

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 1

Виконав
студент групи КН-25-1
Гордієнко В. Ю.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Асимптотична складність алгоритмів. **O-нотація**

Мета: набути практичних навичок у розв'язанні задач на оцінку асимптотичної складності алгоритмів у O .

Порядок виконання роботи

6. Довести, що $f(n) = 150n^2 + 11 = O(n^2)$.

Функція $f(n) = O(g(n))$, якщо існують константи $c > 0$ і n_0 , такі що:

$f(n) \leq c \cdot g(n)$ для всіх $n \geq n_0$;

Підставимо наші значення у формулу, отримаємо: $150n^2 + 11 \leq c \cdot n^2$

При $n \geq 1$ виходить $150n^2 + 11 \leq 150n^2 + 11n^2 = 161n^2$

З цього випливає, що існують константи $c = 161$ і $n_0 = 1$, для яких $150n^2 + 11 \leq c \cdot n^2$, отже $150n^2 + 11 = O(n^2)$.

11. Нехай $f(n) = 75n^2 + 20$ і $g(n) = n^2$. Доведіть, що $f(n) = O(g(n))$.

Функція $f(n) = O(g(n))$, якщо існують константи $c > 0$ і n_0 , такі що:

$f(n) \leq c \cdot g(n)$ для всіх $n \geq n_0$;

Підставимо наші значення у формулу, отримаємо: $75n^2 + 20 \leq c \cdot n^2$

При $n \geq 1$ виходить $75n^2 + 20 \leq 75n^2 + 20n^2 = 95n^2$

З цього випливає, що існують константи $c = 95$ і $n_0 = 1$, для яких $75n^2 + 20 \leq c \cdot n^2$, отже $75n^2 + 20 = O(n^2)$.

Висновки: на цій практичній роботі я набув практичних навичок у розв'язанні задач на оцінку асимптотичної складності алгоритмів у O .

Контрольні питання

1. Що таке асимптотична складність алгоритму?

Це оцінка того, як змінюється час роботи або кількість операцій алгоритму, коли розмір вхідних даних n збільшується.

2. Яким чином визначається O -нотація і яка її сутність?

O -нотація показує верхню межу складності алгоритму. Вона описує, як швидко росте кількість операцій, і ігнорує константи та менші члени.

3. Які основні правила використання O -нотації при аналізі алгоритмів?

Константи відкидаються, залишають тільки найшвидше зростаючий член; у вкладених циклах перемножують складності; у послідовних частинах алгоритму – беруть найбільшу складність.

4. Що означають вирази $O(1)$, $O(n)$, $O(n^2)$ в контексті асимптотичної складності?

$O(1)$ – постійний час, не залежить від n ;

$O(n)$ – час росте пропорційно кількості даних;

$O(n^2)$ – час росте приблизно як квадрат кількості даних (часто через вкладені цикли).

5. Яким чином визначити асимптотичну складність алгоритму за його кодом або математичним виразом?

- Порахувати, скільки разів виконуються основні операції;
- Подивитися на цикли і вкладеність;
- Записати математичний вираз і залишити головний член, записавши його у O -нотації.